From demonstrated work to executable workflows

ABB engineering training guide

Firmware, hardware, and test teams | 15 October 2026 | virtual environment edition

Outcome. Complete a real task with a coding agent, explain your decisions, extract a reusable Markdown workflow, and implement its repeatable activities as clear deterministic code. Prove that the extracted workflow works on a fresh case.

Your four steps

1. Walk through the task in the terminal with GitHub Copilot CLI, explaining why. After completion, ask it to document the workflow.
2. Review and abstract the workflow. Replace incident-specific references with inputs; preserve genuine engineering constraints.
3. Identify operations that do not need an LLM and can become deterministic code.
4. Ask GitHub Copilot CLI to implement highly verbose, readable code for those operations; verify it and update the workflow.

What is included. Six lab guides, shared prompts, local Copilot CLI and Codex setup, virtual environment CI and labeled historical Actions results, an assessment rubric, and an embedded ZIP containing runnable synthetic starters. Starter tools provide basic access to data and systems. Participants create the reusable workflow automation during the labs.

Scope. These exercises use synthetic devices, board data, and a local WebUI. No ABB production repository, device, Azure DevOps project, or proprietary document is required. Firmware lab 2 runs C with the host compiler; hardware calculations use an explicitly idealized circuit; browser labs use Selenium against an actual local application.

Acceptance status. Technical and editorial checks are recorded at the end of this guide. Live Copilot Actions results are recorded in the release record. Local Copilot access and a facilitated learner pilot remain separate checks. The readiness gates make those requirements explicit; this document does not claim an unperformed classroom acceptance.

Start with the route on page 2. Unpack the learner distribution or the starter attachment, then pass the readiness gate.

1 Training route and responsibilities

Participant route: setup -> readiness check -> shared method -> your team's first lab -> your team's second lab -> transfer comparison -> assessment. Read the shared prompts once; lab cards supply only the task-specific details. Host commands start in the unpacked package folder. Each prepared workspace includes its complete current task-card.md. Exercise commands run inside the exact workspace printed on each card; use a fresh host terminal to enter another exercise.

Audience	Required reading and work
Everyone	Sections 2-6: extract materials, configure tools, pass readiness, learn the four steps
Firmware	FW1 device troubleshooting, then FW2 parser repair
Hardware	HW1 board-data reconciliation, then HW2 calculations and calibration
Test	T1 dynamic WebUI automation, then T2 test effectiveness
CI operator	Section 8: use the existing ABB-Training secret and validation workflow
Facilitator	Sections 9-10 and the separate facilitator checks before delivery

Roles. The participant explains decisions and reviews generated work. The coding agent executes agreed operations and writes the extracted artifacts. The facilitator prepares the environment, controls reset copies, checks acceptance evidence, and resolves specification questions. No one should mark a result accepted because the agent says it is complete.

Suggested schedule for each team

Activity	Minutes
Setup and live readiness checks	30
Shared method and first demonstration	15
First lab: stages 1 / 2 / 3 / 4	25 / 15 / 10 / 30
Break	10
Second lab: stages 1 / 2 / 3 / 4	25 / 15 / 10 / 30
Fresh-case replay and debrief	30
Total	245

These are facilitation budgets, not measured completion times. Set up the virtual environment and rehearse the browser before the session. If a stage reaches its budget, record the unfinished requirement; do not skip verification to obtain a pass.

Agent choice. Use GitHub Copilot CLI for guided work, workflow extraction and code generation. Keep the selected Copilot model consistent through all four stages and both replay conditions, and record it with the CLI version. A change of agent/model is a separate intervention, not an effect of adding deterministic code.

2 Set up the Python virtual environment

Unpack the learner distribution or the handbook's **ABB_Workflow_Training_Starters.zip** attachment. Install Python 3.11+ with venv/pip and Git. FW2 needs GCC (use WSL on Windows). Browser labs need Chrome/Chromium and a Selenium driver, which Selenium Manager can download. Install Node.js 22+ with npm for the agents and Playwright browser tools.

Host terminal — Bash, in the package folder:

```
python3 setup_env.py --agents
source .venv/bin/activate
abb doctor --offline
```

Host terminal — Windows PowerShell:

```
py -3 setup_env.py --agents
. .\.venv\Scripts\Activate.ps1
abb doctor --offline
```

See README for host browser libraries and activation alternatives. Work lives under `work/`, run evidence under `runs/`, exports under `export/`. Activate the environment in every new terminal. Preparation refuses existing names; re-enter an existing workspace to resume.

3 Authenticate and verify the coding agents

Host terminal, in the package folder with `.venv` activated:

```
abb login codex
abb login copilot
abb prepare fw1 demo --name agent-smoke
abb agent codex --workspace work/agent-smoke
```

Complete the device flow with the approved account. Credentials persist in ignored `.auth/` provider directories. Ask the agent to run pwd and `git status --short`, explain actual output and change no files. Review normal approvals and perform the corresponding Copilot smoke. Login alone does not prove model access. Keep the same provider/model across the lab and transfers.

Focused context. Deliver the current stage and named inputs while retaining the full task card and acceptance contracts as authority. Scripted CI saves exact prompt/context snapshots and delivery hashes, uses an ephemeral Copilot profile, disables personal custom instructions and skill retrieval, and supplies explicit browser MCP configuration. Classroom work keeps normal approvals and pauses for participant explanation.

4 Pass readiness, resume and export

`abb doctor --offline` checks the active virtual environment, dependencies, available host tools, compilation and clean preparation. Missing tools are BLOCKED; browser execution and accounts are UNTESTED until exercised. Run a T1/T2 browser preflight with its server running and make a real harmless provider tool call before marking those capabilities ready.

Each exercise supplies `abb shell --workspace work/NAME`. Run `pwd` (PowerShell: `Get-Location`) before its commands. `exit` returns to the package folder and preserves work. Reopening a terminal means activating `.venv` and repeating that exercise's entry; preparation is only for fresh work.

For T1/T2, Terminal A runs `python server.py`; Terminal B activates the same environment and enters the same workspace for tests. Preview at `http://127.0.0.1:8765`. Stop your server with Ctrl-C before changing workspaces.

Host terminal — create a fresh submission export:

```
abb export --output export/submission.zip
```

Exports exclude `.env`, `.auth`, provider stores, Git metadata and caches. Inspect code and evidence before submission. Keep FAIL and BLOCKED results meaningful: a starter defect, missing authority, unavailable account and failed browser launch are distinct findings. If a directory is wrong, exit and repeat its workspace entry command.

5 The four-stage method

Use GitHub Copilot CLI for every stage. Keep the guided demonstration in its own conversation, then start a fresh invocation/conversation for abstraction, extraction and implementation. Pass the saved workflow, map and named evidence each stage needs; do not paste the entire prior transcript. Start separate fresh conversations for both transfer conditions. Stage 1 still pauses for the participant's explanation.

Stage 1 Demonstrate and document

First read the current lab card section and its named inputs. Keep the complete task card and acceptance contract available for unresolved requirements. Tell the agent the goal and allowed files. Guide one operation at a time. Before execution, explain purpose and expected observation; after execution, interpret the actual result. Include failed attempts and recovery. Do not ask the agent to solve the entire task autonomously or write the reusable automation yet.

Start prompt

> We will complete this lab together, one terminal operation at a time. Read the task and requirements. For each operation, state its purpose, proposed command, expected evidence, and files it may change. Wait for my engineering explanation before continuing to the next operation. Record actual commands, outputs, decisions, and recovery steps in evidence/session.md. Do not create automation/ at all before Stage 4, even for fixtures or placeholders. Put disposable early fixtures under evidence/stage1-fixtures/. Existing low-level starter tools are allowed. Keep the decision ledger concise and link raw evidence.

Explain reasoning as you work: “I am capturing configuration before changing it so that we can establish what changed.” This explanation becomes part of the material from which the workflow is extracted.

After the task's acceptance checks pass, use the documentation prompt

> Reconstruct the process we actually executed in this conversation. Write workflow.md with its goal, prerequisites, ordered steps, decision branches, expected evidence, failure handling, and completion criteria. Reference evidence/session.md. Distinguish executed operations from ideas we never tried. Do not invent missing steps or claim unobserved results.

Evidence ownership. Original baselines, raw captures and source authorities are immutable. `evidence/session.md` is an append-only journal: preserve previous entries, but allow later entries without invalidating an otherwise valid read-only replay. Host-generated `context/` changes between stages and must not be frozen as an original capture.

Stage 1 exit: the task is completed or explicitly blocked; the record distinguishes commands from intentions; the workflow explains why the operations were performed. If blocked, the facilitator resolves the environment or requirement before the class continues. Do not extract a successful workflow from an incomplete incident.

Stage 2 Abstract the workflow

> Review workflow.md for reuse on another case of this same task. Replace incident-specific paths, IDs, versions, and values with named inputs. Keep genuine protocol constants, engineering limits, source-authority rules, and acceptance criteria. Define every input, its source, validation, and missing-input behavior. Preserve observed branches; label untested branches. Remove anecdotes and duplicate instructions. Save the reviewed result as workflow.md and make a byte-identical checkpoint named workflow-md-only.md. Only refine these documents and stage-required non-secret configuration; do not rerun completed experiments, change task artifacts or create automation/. Carry later implementation obligations forward without implementing them now.

Stage 2 exit: another engineer can identify all required inputs and completion conditions without the original conversation. A concrete limit is either a justified invariant or an explicit input; it is never silently deleted.

6 Convert the repeatable operations into code

Stage 3 Make the extraction decisions

> Review each step in workflow.md. Write extraction-map.md with columns: purpose, inputs, operation, expected output, failure condition, and disposition. Disposition is deterministic code, model judgment, or human decision. Prioritize collection, parsing, calculations, comparisons, validation, and factual reporting. Explain which preconditions code can check. Do not propose code that merely asks another LLM to perform the same operation. Only write the extraction map at this stage; do not rerun the demonstration or implement preservation utilities/automation yet.

Stage 3 exit: every proposed function has a defined input/output contract and a meaningful failure condition. Keep hypotheses, ambiguous requirements, and interpretation of unfamiliar evidence with the model or engineer.

Stage 4 Implement highly verbose code

> Implement the deterministic activities in extraction-map.md under automation/. Use descriptive function and variable names, small functions, type hints where useful, docstrings, explicit intermediate values, and comments explaining engineering intent. Log steps and expected/observed results. Make failures actionable. Do not silently default missing observations or swallow exceptions. Return machine-readable results and meaningful exit codes. Implement feasible prerequisites in code. Keep source-of-truth inputs and instructor checks unchanged. Add checks for the observed defect, a valid case, and the lab's failure branch. Build self-contained fixtures under automation/tests/; their expected values must not depend on which task case is active. Preserve prepared authoritative sources. Run them, inspect the results, and revise until the stated criteria pass. Update workflow.md to call the new entry points and interpret their output; preserve workflow-md-only.md unchanged. Document the exact commands in automation/README.md.

Delivery and repeat-run checks. Complete mandatory entry points, documentation and checks before optional hardening. Keep helper-owned logs/metadata in unique run directories; stdout/stderr redirection belongs to the caller, whose console log may already be open. Never reserve or overwrite an inferred sibling .log file. Run the documented entry point with caller redirection and again with a fresh requested report path. Verify a completed demo without repeating its mutation or rebuilding missing history. Preserve workflow-md-only.md and original captures; later journal entries may append.

Operation-log envelope for learner-created automation

```
{
  "operation": "capture_state",
  "status": "blocked",
  "inputs": {"device_id": "read from actual input"},
  "expected": "all required observations available",
  "observed": "firmware read failed",
  "evidence": ["evidence/actual-read-log.json"],
  "next_action": "restore the read path, then capture again"
}
```

The JSON is a format illustration, not an observed run. Use real input values and evidence paths. Lab automation returns exit 0 for an engineering pass, 1 for an engineering failure, and 2 for blocked inputs or infrastructure errors. The instructor checker can accept a faithful fail or blocked assessment and return 0; that is a separate training decision. This envelope records operations; a lab-specific report schema, such as hardware REPORT_CONTRACT.md, remains authoritative for report.json. These conventions apply to the code participants create, not automatically to starter commands, pytest, GCC, or agent CLI exit codes.

Code review: check arithmetic, units, source authority, subprocess exit status, error handling, and path parameters. Comments and line count are not evidence of correctness. Detailed logs must not contain credentials. A script that validates a prerequisite is a guardrail within that script; it is not a security boundary if the same user can bypass it.

Stage 4 exit: actual tests support the code's behavior; the workflow now invokes it; the participant can explain what moved out of model reasoning and what still needs judgment.

7 Replay and compare the extracted workflows

Do this after completing the two labs for your team. The facilitator selects at least one lab for the paired replay; use both if time permits.

1. Create two fresh transfer workspaces from the same starter case. Use the lab's preparation command with different output names. Keep the original faulty inputs; do not copy repaired demo data or finished task implementations.
2. For **MD-only**, copy workflow-md-only.md into the workspace as workflow.md. Do not include automation/. Ordinary terminal tools and starter utilities remain available.
3. For **MD plus code**, copy the updated workflow.md and automation/. Include only reusable helpers and their documented dependencies; exclude completed demo outputs and evidence.
4. Start a new agent conversation in each workspace. Use the same agent/model and task card. Explicitly ask it to read workflow.md; do not depend on filename auto-discovery.
5. Run the task and compare actual evidence against the same task-specific criteria. Save replay results separately from the teaching conversation.

Replay prompt

> Read workflow.md and the current task requirements. Execute the workflow on this fresh case. Use the inputs in this workspace, not values remembered from a previous run. Record commands, observations, failures, and decisions. Report accepted, unresolved, or blocked with evidence. Do not change authoritative sources or instructor evaluation code. Stop when the task's stated completion condition is reached or a required input cannot be obtained.

Comparison	Record for each version
Correctness	Each acceptance criterion passed, failed, or blocked
Process	Baseline collected; branches followed; missing inputs handled
Reuse	Old IDs, paths, values, or results accidentally carried forward
Execution	Actual helper invocation in code mode; manual fallback is not a code-mode pass
Effort	Elapsed time and facilitator interventions
Claims	Any completion claim unsupported by the recorded evidence

Interpretation. This is a learning exercise about transferring a demonstrated process. One pair of runs does not establish statistical superiority. If the MD-only agent independently writes useful scripts, record that behavior; do not prohibit it to force a desired result.

Fresh execution boundary. Each replay uses only its current workspace and the supplied workflow/helper package. Repair its fresh parser/POM/tests; do not copy demo implementations, results or assessments. Code-assisted replay must actually invoke the supplied helpers and retain their feedback. A faithfully reported FAIL/BLOCKED engineering outcome remains valid when the task requires it.

Required submission per lab: workflow-md-only.md, workflow.md, extraction-map.md, automation/ with its checks and README, and evidence/ containing the actual demonstration and verification records. A blocked transfer can be the correct outcome when the task deliberately omits required authority.

FW1 Capture state before troubleshooting

Task card. Restore communication with the simulated device. Explain the cause using observations collected before the change, make one justified configuration change, and verify recovery. Do not repair the simulator's implementation.

Fresh demonstration entry Host terminal — in the unpacked package folder; fresh FW1 demonstration:

```
abb doctor --offline
abb prepare fw1 demo
abb shell --workspace work/fw1-demo
```

Inside the workspace:

```
pwd
# Expected: work/fw1-demo
```

This opens a local shell with the active virtual environment. Agent authentication is separate. If your chosen provider is not already authenticated, use its `abb login` command from the setup instructions. Keep the same agent/model throughout the exercise.

Stages 1–4 stay in this shell. To resume, return to the package folder, activate `.venv`, and repeat the exact workspace entry command without preparation. Check `pwd` before continuing. Shell exit preserves your files; choose a fresh name for another attempt.

Read `requirements.md`. Start the agent here and use the Stage 1 start prompt. The state is already initialized under `device/`. Reinitialization is unnecessary and refuses existing state.

Allowed changes: device configuration through `fw1/device.py set`, plus the common workflow, automation, and evidence files. Do not edit `device/state.json`, the simulator, or `fw1/expected.json` directly.

Stage 1 Guided operations

Run these one at a time, explaining the question each answers: **Inside the workspace**:

```
python fw1/device.py --workdir device read --field identity
python fw1/device.py --workdir device read --field firmware
python fw1/device.py --workdir device read --field config
python fw1/device.py --workdir device read --field faults
python fw1/device.py --workdir device logs
python fw1/device.py --workdir device probe
python -m json.tool fw1/expected.json
```

Decision point	Explanation the participant must supply
Before any write	Which observations constitute the baseline, and where will they be saved?
After the probe	What supports a configuration hypothesis rather than a software repair?
Before changing baud	Which expected entry belongs to the observed device identity?
After the change	Which current observations demonstrate recovery?

Use the tool's help to construct a single `set --parameter baud --value` operation from the observed reference. Save pre-change evidence before executing it. Then repeat configuration, faults, and probe. Historical logs may retain the old incident; explain why that differs from an active fault.

Demonstration complete: identity-matched reference used; baseline retained; one recorded configuration change; current fault cleared and probe successful. Now use the shared documentation prompt, then continue on the next page.

FW1 Extract the reusable operations

Stay in the existing work/fw1-demo workspace shell for Stages 2–4. If reopening a terminal, repeat this lab's demo entry command without preparation.

Stage 2 Abstraction focus

Inputs are the device tool path, state-directory path, expected-configuration path, and evidence destination. Resolve the expected settings from the observed identity. Do not carry the demo device ID or baud value into the reusable workflow.

Stage 3 Deterministic boundary

Move into code	Retain as reasoning
Capture identity, firmware, configuration, faults, logs, and probe output	Decide which evidence supports the fault hypothesis
Validate required observations and identity/reference match	Decide whether an unfamiliar fault warrants a different investigation
Compare before/after values and probe status	Explain why the observed change demonstrates recovery

Stage 4 Required implementation behavior

Create `automation/capture_state.py` and a controlled experiment entry point with documented arguments. Capture must invoke the low-level tool and retain complete raw outputs, commands, exit status, timestamps and input identity by default. Save the before-capture before any mutation; failed evidence persistence blocks the write. It must fail if any required read fails; a partial capture cannot become a baseline marked complete.

The experiment operation must require a valid baseline for the same device and compare the current configuration with that baseline before writing. If they differ, return blocked and request a new capture. It must apply at most one allowed parameter change, then capture and compare again. Preserve original before/after and mutation records on repeat runs. This is enforcement inside the learner's entry point; direct simulator access remains available in this teaching fixture.

Journal check: append a legitimate session entry, rerun read-only verification and confirm original captures and old journal entries remain intact. Do not freeze the whole growing journal hash.

Required checks: valid capture; missing/nonexistent state directory produces blocked; unknown identity cannot select a reference; experiment without a baseline is rejected; stale baseline is rejected; correct change restores the probe. Use disposable prepared cases for checks that mutate state.

Fresh transfer entries after Stage 4 After Stage 4, stop any owned app server and use two fresh transfer workspaces. Prepare each once. Copy only the workflow or reusable automation specified by the exercise, keeping the demonstration evidence in its original workspace.

Host terminal — fresh instructions-only transfer:

```
abb doctor --offline
abb prepare fw1 transfer --name fw1-transfer-md
abb shell --workspace work/fw1-transfer-md
```

Inside the workspace:

```
pwd
# Expected: work/fw1-transfer-md
```

Host terminal — fresh instructions-plus-code transfer:

```
abb doctor --offline
abb prepare fw1 transfer --name fw1-transfer-code
abb shell --workspace work/fw1-transfer-code
```

Inside the workspace:

```
pwd
# Expected: work/fw1-transfer-code
```

To resume either transfer, repeat its host-side entry command without `abb prepare`. Do not overwrite an existing workspace.

Use the shared replay procedure. The transfer has a different identity and required baud. The learner's code must discover both from current evidence. Do not copy `device/` from the demo.

Assessment: the facilitator checks actual before/after records, the device-matched reference, and a fresh successful probe. Automated starter smoke tests validate the fixture, not the learner's reasoning or baseline capture.

Debrief: which terminal operations stopped requiring the model to reconstruct commands? Which diagnosis decisions still depended on engineering interpretation?

FW2 Reproduce and repair a streaming parser

Task card. Repair the supplied C parser to satisfy its signed-value and streaming requirements. Preserve its public interface. Demonstrate the failures, explain the repair, and extract a reusable build/reproduction/verification workflow.

Fresh demonstration entry Host terminal — in the unpacked package folder; fresh FW2 demonstration:

```
abb doctor --offline
abb prepare fw2 demo
abb shell --workspace work/fw2-demo
```

Inside the workspace:

```
pwd
# Expected: work/fw2-demo
```

This opens a local shell with the active virtual environment. Agent authentication is separate. If your chosen provider is not already authenticated, use its `abb login` command from the setup instructions. Keep the same agent/model throughout the exercise.

Stages 1–4 stay in this shell. To resume, return to the package folder, activate `.venv`, and repeat the exact workspace entry command without preparation. Check `pwd` before continuing. Shell exit preserves your files; choose a fresh name for another attempt.

Read `requirements.md` before editing. A packet is `A5 02 sequence low high checksum`; the value is signed 16-bit little-endian and checksum is XOR of the first five bytes. Each CLI argument is one receive chunk in the same stream. The complete contract defines noise, invalid length, checksum failure, and resynchronization. There is no timeout requirement in this fixture.

Allowed changes: `fw2/parser.c`, plus common workflow/automation/evidence files. Preserve `parser.h`, `main.c`, and the specification. The compiled `fw2/parser` is a generated artifact.

Stage 1 Guided operations

Inside the workspace:

```
gcc -std=c11 -Wall -Wextra -Werror \
  fw2/parser.c fw2/main.c -o fw2/parser
./fw2/parser a5020785ffda
./fw2/parser a50207 85ffda
```

Both invocations describe sequence 7 and value -123. Save their actual outputs. Explain why receive-chunk boundaries must not change the decoded result. Inspect the integer conversion and lifetime of partial parser state before changing code.

Make a focused repair. Rebuild and repeat the same reproductions. Then run a positive value, negative boundary, checksum error, and a valid packet following noise. Derive expected values from the specification, not the current output.

Decision point	Evidence to explain
Build passes, behavior fails	Why successful compilation does not establish protocol correctness
Whole versus split packet	Whether state survives between feed calls
Signed conversion	How the same two bytes represent a negative two's-complement value
After repair	Which additional cases protect against a narrow example-specific fix

Demonstration complete: both reproductions produce the specified result; the facilitator's protocol check passes; the participant explains the change. Use the shared documentation prompt.

FW2 Extract build and replay tooling

Stay in the existing work/fw2-demo workspace shell for Stages 2–4. If reopening a terminal, repeat this lab's demo entry command without preparation.

Stage 2 Abstraction focus

Parameterize source directory, compiler command, executable path, packet fields, chunk schedule, and evidence destination. Preserve the six-byte packet format, signed range, checksum definition, and streaming semantics as specification invariants.

Stage 3 Deterministic boundary

Move into code	Retain as reasoning
Compile and capture exit status/stdout/stderr	Locate the implementation defect
Construct packets from valid fields and split by supplied boundaries	Decide how to repair state handling without unnecessary changes
Replay streams and compare actual/expected objects	Resolve any future ambiguous protocol requirement
Produce requirement-linked verification records	Explain why the check set is adequate for this task

Stage 4 Required implementation behavior

Create readable build, packet construction, stream replay, and comparison functions under `automation/`. Do not put a corrected parser inside the workflow automation. The automation verifies the candidate parser; the agent still performs task-specific repairs.

Represent compilation failure separately from an incorrect decode. A failed compiler command must stop execution of an older executable. Every result must identify the source snapshot or digest, command, input bytes/chunks, expected objects, actual objects, and process result.

Required checks: whole/split equivalence; signed minimum and maximum; bad checksum rejected; incomplete tail emits nothing; valid frame after noise recovered; compilation failure cannot reuse a stale executable. Follow the exact resynchronization contract in requirements.md.

Fresh transfer entries after Stage 4 After Stage 4, stop any owned app server and use two fresh transfer workspaces. Prepare each once. Copy only the workflow or reusable automation specified by the exercise, keeping the demonstration evidence in its original workspace.

Host terminal — fresh instructions-only transfer:

```
abb doctor --offline
abb prepare fw2 transfer --name fw2-transfer-md
abb shell --workspace work/fw2-transfer-md
```

Inside the workspace:

```
pwd
# Expected: work/fw2-transfer-md
```

Host terminal — fresh instructions-plus-code transfer:

```
abb doctor --offline
abb prepare fw2 transfer --name fw2-transfer-code
abb shell --workspace work/fw2-transfer-code
```

Inside the workspace:

```
pwd
# Expected: work/fw2-transfer-code
```

To resume either transfer, repeat its host-side entry command without `abb prepare`. Do not overwrite an existing workspace.

The fresh starters retain the defects. Use this new reproduction after building: **Inside the workspace**:

```
./fw2/parser a502 2a00 f875
```

Expected result: sequence 42, value -2048. Transfer the workflow and helpers, not the repaired `parser.c` or old executable. The new agent must use the process to repair the fresh candidate.

Facilitator protocol check, from the training root after a learner build: **Host terminal — facilitator package only**:

```
python training_kit/firmware/instructor/smoke.py work/fw2-demo/fw2/parser
```

The facilitator's independent checks and assessment sources are supplied separately from the learner materials.

Use the transfer executable path for a transfer submission. The checker evaluates 50 stream cases plus malformed input. Also review the explanation and automation; a protocol pass alone does not complete the training assessment.

HW1 Reconcile a board data package

Task card. Reconcile a generated netlist with the approved BOM and schematic export. Preserve source authority, distinguish equivalent units from real discrepancies, and report every change with evidence.

Fresh demonstration entry Host terminal — in the unpacked package folder; fresh HW1 demonstration:

```
abb doctor --offline
abb prepare hw1 demo
abb shell --workspace work/hw1-demo
```

Inside the workspace:

```
pwd
# Expected: work/hw1-demo
```

This opens a local shell with the active virtual environment. Agent authentication is separate. If your chosen provider is not already authenticated, use its `abb login` command from the setup instructions. Keep the same agent/model throughout the exercise.

Stages 1–4 stay in this shell. To resume, return to the package folder, activate `.venv`, and repeat the exact workspace entry command without preparation. Check `pwd` before continuing. Shell exit preserves your files; choose a fresh name for another attempt.

Inputs: `source/approval.json` declares the expected board/revision; `source/bom.csv` is authoritative for component identity and value; `source/schematic.json` is authoritative for pin-to-net mapping and corroborates values. `derived/netlist.json` is an output to review, not an authority. Required source provenance must be present and consistent.

Allowed changes: `derived/netlist.json`, `report.json`, and common workflow/automation/evidence files. Preserve source files and any supplied tools. Use the report contract in `REPORT_CONTRACT.md` supplied in the prepared workspace.

Stage 1 Guided operations

Inside the workspace:

```
python -m json.tool source/approval.json
cat source/bom.csv
python -m json.tool source/schematic.json
python -m json.tool derived/netlist.json
```

Before correcting anything, record the source inventory and explain authority. Compare components by reference designator and pins by pin number, not file order. Normalize values by dimension: kOhm to ohms, nF and uF to farads. Treat IC part identifiers as literal strings.

Decision point	Required reasoning
Revision checks	Is every required source independently identified as belonging to the approved revision?
Different notation	Are the values physically equivalent after unit conversion?
Conflicting source facts	Is there enough authoritative evidence to correct the export?
Proposed correction	Which source field supports this exact change?

Correct only demonstrable export errors. Re-read the result and create report.json with findings, evidence references, assumptions, and changed paths. A resolved discrepancy remains in findings so the change is explainable.

Demonstration complete: real value and pin discrepancies corrected; equivalent notation not reported as an error; sources unchanged; report supported by actual observations. Ask for the workflow document.

HW1 Extract source and consistency checks

Stay in the existing work/hw1-demo workspace shell for Stages 2–4. If reopening a terminal, repeat this lab's demo entry command without preparation.

Stage 2 Abstraction focus

Parameterize the package root, source-role paths, expected board/revision declaration, and output paths. Do not embed the demo's component references. Preserve the rule that an expected revision is not evidence for a missing revision on a source document.

Stage 3 Deterministic boundary

Move into code	Retain as reasoning
Enumerate required files and validate metadata	Resolve genuinely conflicting approved sources
Parse and normalize typed quantities	Interpret an ambiguous native drawing in a future extension
Match components and compare values/pin mappings	Decide which authority must supply missing evidence
Produce field-linked discrepancy records	Review whether the proposed source hierarchy fits a real project

Stage 4 Required implementation behavior

Create a reusable board-package checker. Separate inventory/provenance validation from numerical and connectivity comparison. If provenance fails, return blocked before modifying derived data. Make unit conversion explicit and reject unknown units rather than guessing.

Include functions for component indexing, dimensional normalization, pin comparison, and report generation. If the workflow also applies corrections, require a complete discrepancy plan and preserve a before/after record. Rechecking the corrected package must find no unresolved discrepancy.

Required checks: equivalent 100 nF and 0.1 uF; incorrect resistor value; incorrect pin map; reordered components; missing source revision; missing component. Keep report.json compatible with REPORT_CONTRACT.md.

Fresh transfer entries after Stage 4 After Stage 4, stop any owned app server and use two fresh transfer workspaces. Prepare each once. Copy only the workflow or reusable automation specified by the exercise, keeping the demonstration evidence in its original workspace.

Host terminal — fresh instructions-only transfer:

```
abb doctor --offline
abb prepare hw1 transfer --name hw1-transfer-md
abb shell --workspace work/hw1-transfer-md
```

Inside the workspace:

```
pwd
# Expected: work/hw1-transfer-md
```

Host terminal — fresh instructions-plus-code transfer:

```
abb doctor --offline
abb prepare hw1 transfer --name hw1-transfer-code
abb shell --workspace work/hw1-transfer-code
```

Inside the workspace:

```
pwd
# Expected: work/hw1-transfer-code
```

To resume either transfer, repeat its host-side entry command without `abb prepare`. Do not overwrite an existing workspace.

The package changes board identity, component references/order, and omits one source revision. No recovery evidence is supplied. A correct transfer stops with a blocked assessment, requests an explicitly revisioned approved source, and leaves the derived netlist unchanged.

Facilitator check, from the training root: **Host terminal — facilitator package only**:

```
python training_kit/hardware/instructor/check.py --lab hw1 --case demo --workspace
work/hw1-demo
```

The facilitator's independent checks and assessment sources are supplied separately from the learner materials.

For transfer, change the case and workspace arguments. The instructor check accepts a faithful blocked report; that does not mean the board data has been verified.

Debrief: did code remove comparison work while preserving the responsibility to establish source authority?

HW2 Calculate limits and refresh calibration

Task card. Review an idealized voltage-divider input against its ADC voltage limit, then regenerate stale nominal calibration from the current inputs. Report the design assessment independently of whether the file generator ran successfully.

Fresh demonstration entry Host terminal — in the unpacked package folder; fresh HW2 demonstration:

```
abb doctor --offline
abb prepare hw2 demo
abb shell --workspace work/hw2-demo
```

Inside the workspace:

```
pwd
# Expected: work/hw2-demo
```

This opens a local shell with the active virtual environment. Agent authentication is separate. If your chosen provider is not already authenticated, use its `abb login` command from the setup instructions. Keep the same agent/model throughout the exercise.

Stages 1–4 stay in this shell. To resume, return to the package folder, activate `.venv`, and repeat the exact workspace entry command without preparation. Check `pwd` before continuing. Shell exit preserves your files; choose a fresh name for another attempt.

Inside the workspace:

```
python -m json.tool source/circuit.json
python -m json.tool derived/calibration.json
```

Model contract: ideal source; upper resistor Rt and lower resistor Rb; nominal capacitor C from divider node to ground; ideal high-impedance ADC. Independent resistor tolerances are +/-t. No ADC loading, sampling transient, leakage, temperature drift, or capacitor tolerance is included. Source values use SI units. These assumptions bound the exercise.

Allowed changes: `derived/calibration.json`, `report.json`, and common artifacts. Preserve source/circuit.json and tools/. Use REPORT_CONTRACT.md for report fields and precision.

Stage 1 Guided operations

Calculate one quantity at a time in the terminal, explaining the equation before running it. Read values from the actual source, even when using a short Python expression.

Quantity	Required equation
Nominal gain	Rb / (Rt + Rb)
Maximum gain	Rb*(1+t) / (Rt*(1-t) + Rb*(1+t))
Nominal / maximum ADC voltage	Maximum input voltage multiplied by the corresponding gain
Equivalent resistance	Rt*Rb / (Rt+Rb)
Nominal cutoff	1 / (2*pi*equivalent resistance*C)
Inverse gain	(Rt+Rb) / Rb
ADC margin	ADC limit minus maximum ADC voltage

Explain why upper-resistor minimum and lower-resistor maximum maximize output. Compare the existing coefficient and provenance with current inputs. Then invoke the existing generator: **Inside the workspace**:

```
python tools/generate_calibration.py --config source/circuit.json \
  --output derived/calibration.json --replace
```

Demonstration complete: all eight quantities recorded, voltage limit evaluated, calibration regenerated and checked, report.json written with assumptions and evidence. Now document the observed workflow.

HW2 Extract calculations and freshness checks

Stay in the existing work/hw2-demo workspace shell for Stages 2–4. If reopening a terminal, repeat this lab's demo entry command without preparation.

Stage 2 Abstraction focus

All component values, tolerance, maximum input, ADC limit, and file paths are inputs. Equations and model assumptions remain explicit. Do not turn the demo's coefficient into a universal constant.

Stage 3 Deterministic boundary

Move into code	Retain as reasoning
Validate positive finite values, units, and allowed tolerance	Decide whether the ideal circuit model applies to a real design
Calculate quantities and intermediate values	Investigate behavior outside the model's scope
Compare worst-case voltage with the limit	Select a new design when a limit is exceeded
Regenerate calibration and compare source digest	Approve changes to engineering assumptions

Stage 4 Required implementation behavior

Create a calculation entry point that accepts a source JSON path and report destination. Give each equation a named function or clearly named intermediate values. Log units, substituted values, calculated results, source revision, and limits. Use the existing calibration generator; do not recreate its file format inconsistently.

Validate freshness using the canonical source digest specified in REPORT_CONTRACT.md. A design-limit failure must remain visible even if nominal calibration was regenerated correctly. The exercise asks for assessment of the supplied design, not redesign to force a pass.

Required checks: valid demo; worst-case limit violation; invalid/missing input; changed input invalidates old calibration provenance; regeneration is repeatable for identical input. The source JSON must remain unchanged.

Fresh transfer entries after Stage 4 After Stage 4, stop any owned app server and use two fresh transfer workspaces. Prepare each once. Copy only the workflow or reusable automation specified by the exercise, keeping the demonstration evidence in its original workspace.

Host terminal — fresh instructions-only transfer:

```
abb doctor --offline
abb prepare hw2 transfer --name hw2-transfer-md
abb shell --workspace work/hw2-transfer-md
```

Inside the workspace:

```
pwd
# Expected: work/hw2-transfer-md
```

Host terminal — fresh instructions-plus-code transfer:

```
abb doctor --offline
abb prepare hw2 transfer --name hw2-transfer-code
abb shell --workspace work/hw2-transfer-code
```

Inside the workspace:

```
pwd
# Expected: work/hw2-transfer-code
```

To resume either transfer, repeat its host-side entry command without `abb prepare`. Do not overwrite an existing workspace.

The upper resistor changes. Use current inputs for every calculation. Correct transfer behavior produces a design assessment of fail when the worst-case voltage exceeds the stated limit, while recording accurate recalculation and calibration provenance.

Facilitator check, from the training root: **Host terminal — facilitator package only**:

```
python training_kit/hardware/instructor/check.py --lab hw2 --case demo --workspace
work/hw2-demo
```

The facilitator's independent checks and assessment sources are supplied separately from the learner materials.

Change case/workspace for transfer. The check validates numerical results and artifact provenance; it does not certify physical hardware. Compare the documented assumptions and engineering explanation as part of assessment.

Debrief: which parts of the calculation previously consumed model reasoning despite having a fully specified deterministic answer?

T1 Explore WebUI with Playwright MCP

Task card. Use the chosen agent with Playwright MCP to inspect a running dynamic form, then extend the existing Selenium Page Objects and pytest tests. This replaces terminal-driven Selenium inspection; Selenium remains the test execution framework.

Fresh demonstration entry Host terminal — in the unpacked package folder; fresh T1 demonstration:

```
abb doctor --offline
abb prepare t1 demo
abb shell --workspace work/t1-demo
```

Inside the workspace:

```
pwd
# Expected: work/t1-demo
```

This opens a local shell with the active virtual environment. Agent authentication is separate. If your chosen provider is not already authenticated, use its `abb login` command from the setup instructions. Keep the same agent/model throughout the exercise.

Stages 1–4 stay in this shell. To resume, return to the package folder, activate `.venv`, and repeat the exact workspace entry command without preparation. Check `pwd` before continuing. Shell exit preserves your files; choose a fresh name for another attempt.

Inside the workspace — Terminal A: start only this workspace's server and leave it running.

```
python server.py --variant correct --port 8765 --state state.json
```

Host terminal — Terminal B, opened separately in the same package folder:

```
abb shell --workspace work/t1-demo
```

Inside the workspace — Terminal B:

```
pwd
# Expected: work/t1-demo
```

Run the bootstrap/tests from Terminal B. Both terminals enter the same `training` service and workspace. Stop your Terminal A server with Ctrl-C before changing workspaces; if port 8765 is occupied, identify its owning terminal and stop that server. Keep the evidence and do not stop another participant's process.

Keep the server running. In terminal B, repeat the exact Terminal B workspace-entry command for work/t1-demo, and run `python -m pytest browser_preflight.py -q`, then `python -m pytest -q`. Expect a passing preflight, one smoke pass and one skipped placeholder. Browser startup failure is BLOCKED; the skip is unfinished coverage.

Session preparation. Follow the next page before exploration. The supplied synthetic app has no login; use its no-login session bootstrap. To exercise authentication, the facilitator supplies an authorized test URL, account, approved login selectors and an authenticated readiness signal. Do not add login to the protected fixture. Use the chosen agent consistently in the demonstration and both transfer conditions; record versions and model.

Allowed changes. Extend pages/settings_page.py, tests/test_settings.py and workflow/automation/evidence files; add bootstrap and non-secret configuration files. Preserve server, BasePage, conftest.py, case.json and REQUIREMENTS.md. The fixture resets state before each test. For an external target, agree its independent requirements and adapter first; local graders do not validate that target.

Stage 1 Guided operations

1. Read REQUIREMENTS.md, case.json, BasePage, SettingsPage and existing tests. Identify helpers to reuse. Explain the purpose of each action before running it.

2. With the chosen agent, use Playwright MCP to open the target and inspect live snapshots; inspect DOM attributes when needed. Record action, resulting fields, visibility/enabled/required state, readiness signal and proposed Selenium locator. Separate observations from requirements. Snapshot references are temporary, not test locators.

3. Switch Serial to TCP and back, and None to Password and back. Wait for each asynchronous transition. Check locator uniqueness in each relevant state; prefer stable IDs/test attributes or scoped semantic selectors supported by the existing framework.

4. Extend SettingsPage only for the demonstrated interactions. Keep behavioral assertions in tests and readiness checks in Page Objects. Do not replace Selenium with Playwright-generated tests.

5. Save case.json values, wait for completion, reload and assert exact restored active values. Reject an empty password and an invalid port; reload after each rejected save and prove the last valid configuration survived.

6. Run pytest through the actual framework; retain useful failure evidence. Remove the placeholder only after substantive coverage exists. MCP click-through success alone does not establish test coverage. Document the demonstrated workflow before extracting automation.

Demonstration complete: UI-01, UI-02, SAVE-01 and ASYNC-01 have assertions; both switch directions work; values survive reload; no required skips or fixed sleeps. Retain sanitized MCP observations and actual pytest results in evidence/session.md.

T1 Session bootstrap and agent handover

Replacement focus. Replace generic browser-helper extraction with a deterministic local JavaScript bootstrap, authenticated-session handover and reusable execution/evidence helpers. The script runs in Node.js; never serve credentials in browser JavaScript.

Stage 2 Abstract the session workflow

Parameterize target URL, login selectors, authenticated readiness signal, case path and evidence destination. Store test credentials in a local `.env`: `TEST_BASE_URL`, `TEST_USERNAME`, `TEST_PASSWORD`. Commit only an `.env.example` with empty credential values. Before creating secrets, add these rules to the training workspace's `.gitignore`:

```
.env
.env.*
!.env.example
.auth/
```

Check `git check-ignore .env .auth/state.json` and `git ls-files -- .env .auth`. Ignored files must not already be tracked. Git ignore prevents accidental tracking; it does not hide files from the agent or protect local access. Restrict local permissions and exclude secrets/state from prompts, logs, screenshots and archives. Have the operator populate .env without showing its contents to Copilot.

Stage 3 Demonstrate the deterministic boundary

The prepared T1 workspace supplies the reviewed no-login bootstrap under `tools/`; it creates no learner `automation/` code. In Terminal B, with Terminal A's server running:

Inside the workspace:

```
node tools/session_bootstrap.mjs --mode no-login --url http://127.0.0.1:8765 --state
.auth/state.json --report evidence/bootstrap.json
```

The launcher configures actual stdio MCP from the baked tooling when you start `abb agent` in this workspace. It loads `.auth/state.json` into a fresh isolated context. For an authorized external login target, a reviewed adapter must be supplied separately; the supplied script returns BLOCKED in login mode. Do not treat no-login state as authentication. Learner automation is created only after the later abstraction and extraction stages.

Start the chosen provider through `abb agent` after the state file exists. The launcher writes fresh workspace-scoped MCP configuration with the pinned baked Node entry point, sandboxed Chrome and isolated state loading. It performs no package download. Ask the actual MCP tools to reach the form before exploration. Saved state initializes a new context; it does not transfer an open tab, and the local no-login fixture does not prove authentication. Targets requiring other state need a separately reviewed adapter. [M1, M2]

Stage 4 Implement and verify reuse

Implement bootstrap and pytest execution/evidence helpers under automation/; retain the existing Selenium framework. Missing inputs, rejected login, expired state or unreachable URL must report BLOCKED without secrets. Keep assertions in tests. Record actual browser identity, URL, collected/skipped counts and outcomes; distinguish startup failures from assertion failures. Retain a sanitized intentional missing-element screenshot and error context in evidence/. Selenium tests use their own reviewed authentication fixture for external targets; Playwright state is not automatically a Selenium session.

Fresh transfer entries after Stage 4 After Stage 4, stop any owned app server and use two fresh transfer workspaces. Prepare each once. Copy only the workflow or reusable automation specified by the exercise, keeping the demonstration evidence in its original workspace.

Host terminal — fresh instructions-only transfer:

```
abb doctor --offline
abb prepare t1 transfer --name t1-transfer-md
abb shell --workspace work/t1-transfer-md
```

Inside the workspace:

```
pwd
# Expected: work/t1-transfer-md
```

Inside the workspace — Terminal A:

```
python server.py --variant correct --port 8765 --state state.json
```

Host terminal — Terminal B:

```
abb shell --workspace work/t1-transfer-md
```

Inside the workspace — Terminal B:

```
pwd
# Expected: work/t1-transfer-md
python -m pytest browser_preflight.py -q
```

Run this transfer's tests in Terminal B. Stop this Terminal A server before the next workspace; state from the demonstration is not transferred.

Host terminal — fresh instructions-plus-code transfer:

```
abb doctor --offline
abb prepare t1 transfer --name t1-transfer-code
abb shell --workspace work/t1-transfer-code
```

Inside the workspace:

```
pwd
# Expected: work/t1-transfer-code
```

Inside the workspace — Terminal A:

```
python server.py --variant correct --port 8765 --state state.json
```

Host terminal — Terminal B:

```
abb shell --workspace work/t1-transfer-code
```

Inside the workspace — Terminal B:

```
pwd
# Expected: work/t1-transfer-code
python -m pytest browser_preflight.py -q
```

Run this transfer's tests in Terminal B. Stop this Terminal A server before the next workspace; state from the demonstration is not transferred.

To resume either transfer, repeat its host-side entry command without `abb prepare`. Do not overwrite an existing workspace.

Transfer and acceptance. Follow Section 7. MD-only starts with clean Page Objects/tests plus extracted Markdown; code-assisted additionally copies only reusable automation and documented dependencies. Supply credentials locally and bootstrap a fresh session in each; never copy auth state, demo tests or finished Page Objects. Run one workspace at a time on port 8765. Verify both visibility directions and exact current-case persistence. Test successful handover, wrong URL, missing credentials in login mode, invalid/expired authentication and useful element-failure evidence. Mark unexercised login paths BLOCKED. T2 still checks defect detection. Prior CI results cover the Selenium baseline, not this new MCP/login route; rehearse it before delivery.

M1 https://github.com/microsoft/playwright-mcp

M2 https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/add-mcp-servers

T2 Test whether the suite detects defects

Stop the previous T1 or transfer server with Ctrl-C before starting this lab.

Task card. Improve a passing but weak suite so that it detects a persistence defect and validates the specified port boundaries. Demonstrate both correct-app success and relevant defective-app failure.

Fresh demonstration entry Host terminal — in the unpacked package folder; fresh T2 demonstration:

```
abb doctor --offline
abb prepare t2 demo
abb shell --workspace work/t2-demo
```

Inside the workspace:

```
pwd
# Expected: work/t2-demo
```

This opens a local shell with the active virtual environment. Agent authentication is separate. If your chosen provider is not already authenticated, use its `abb login` command from the setup instructions. Keep the same agent/model throughout the exercise.

Stages 1–4 stay in this shell. To resume, return to the package folder, activate `.venv`, and repeat the exact workspace entry command without preparation. Check `pwd` before continuing. Shell exit preserves your files; choose a fresh name for another attempt.

Inside the workspace — Terminal A: start only this workspace's server and leave it running.

```
python server.py --variant correct --port 8765 --state state.json
```

Host terminal — Terminal B, opened separately in the same package folder:

```
abb shell --workspace work/t2-demo
```

Inside the workspace — Terminal B:

```
pwd
# Expected: work/t2-demo
```

Run the bootstrap/tests from Terminal B. Both terminals enter the same `training` service and workspace. Stop your Terminal A server with Ctrl-C before changing workspaces; if port 8765 is occupied, identify its owning terminal and stop that server. Keep the evidence and do not stop another participant's process.

Use terminal B in the same workspace with the training venv active. Run the browser preflight, then `python -m pytest -q`. The initial single smoke test is intentionally weak.

Allowed changes: the same Page Object and test files as T1, plus common artifacts. Preserve the application and independent requirements. Use requirements to determine expected behavior; inspecting the server's fault-switch implementation is not part of the diagnosis.

Stage 1 Guided operations

1. Read the existing assertion and explain what a pass does and does not establish.
2. Read SAVE-01, VAL-01, and ASYNC-01 in REQUIREMENTS.md. Map each to observable browser behavior.
3. Show a valid save and browser reload on the correct app.

4. Stop terminal A with Ctrl-C. With the server stopped, reset the local state and launch the demonstration defect: **Inside the workspace**:

```
python -c "from pathlib import Path; Path('state.json').unlink(missing_ok=True)"
python server.py --variant no-persist --port 8765 --state state.json
```

5. Run the unchanged weak test. Explain why it passes despite the defect.
6. Add assertions for persistence and boundary handling. Use the existing framework and explicit waits.
7. Confirm the strengthened suite detects the demonstrated defect. Restart the correct variant with clean state and confirm the suite passes there.

Required cases: valid TCP ports 1 and 65535; invalid 0, 65536, and non-integer text; invalid save preserves the previous valid configuration; saved active values survive browser reload.

Demonstration complete: correct application passes; the no-persist variant fails because a saved value is not restored; evidence connects the failure to SAVE-01. A crash or unrelated timeout does not count as detecting the defect. Now document the process.

T2 Extract repeatable test effectiveness evaluation

Stay in the existing work/t2-demo workspace shell for Stages 2–4. If reopening a terminal, repeat this lab's demo entry command without preparation.

Stage 2 Abstraction focus

Parameterize the test workspace, application command/variant, port, state path, case data, and evidence destination. Preserve the rule that the suite must pass a correct application before a defective-application failure earns credit.

Stage 3 Deterministic boundary

Move into code	Retain as reasoning
Start/stop the local app and reset owned test state	Judge whether a defect represents a requirement violation
Check server/browser health and run collected tests	Identify which missing assertion allowed the defect to survive
Record counts, exit codes, and failure details	Decide whether the failure is relevant rather than incidental
Compare correct and defective execution records	Improve the test's semantic coverage

Stage 4 Required implementation behavior

Implement a serial evaluation runner that owns only its child server process, waits for readiness, runs the test suite, and stops that process in a finally block. Never terminate unrelated processes merely because they occupy a port. Use separate state files or clean resets for each run.

Record the exact application variant, case, test files/digests, collected/skipped counts, browser startup outcome, and assertion details. A failed startup is blocked. A suite that fails the correct app cannot receive a successful detection score. Require review of the assertion that detected each defect.

Output ownership: the caller owns its console capture; helper-owned logs use fresh run directories. An already-open caller log is not missing/invalid input. Preserve the real pytest status and JUnit output.

Required checks: correct app passes; both required mutants fail through relevant assertions; required skips rejected; occupied port or failed startup reported as infrastructure failure; test failures preserved in the final result rather than swallowed.

Fresh transfer entries after Stage 4 After Stage 4, stop any owned app server and use two fresh transfer workspaces. Prepare each once. Copy only the workflow or reusable automation specified by the exercise, keeping the demonstration evidence in its original workspace.

Host terminal — fresh instructions-only transfer:

```
abb doctor --offline
abb prepare t2 transfer --name t2-transfer-md
abb shell --workspace work/t2-transfer-md
```

Inside the workspace:

```
pwd
# Expected: work/t2-transfer-md
```

Inside the workspace — Terminal A:

```
python server.py --variant correct --port 8765 --state state.json
```

Host terminal — Terminal B:

```
abb shell --workspace work/t2-transfer-md
```

Inside the workspace — Terminal B:

```
pwd
# Expected: work/t2-transfer-md
python -m pytest browser_preflight.py -q
```

Run this transfer's tests in Terminal B. Stop this Terminal A server before the next workspace; state from the demonstration is not transferred.

Host terminal — fresh instructions-plus-code transfer:

```
abb doctor --offline
abb prepare t2 transfer --name t2-transfer-code
abb shell --workspace work/t2-transfer-code
```

Inside the workspace:

```
pwd
# Expected: work/t2-transfer-code
```

Inside the workspace — Terminal A:

```
python server.py --variant correct --port 8765 --state state.json
```

Host terminal — Terminal B:

```
abb shell --workspace work/t2-transfer-code
```

Inside the workspace — Terminal B:

```
pwd
# Expected: work/t2-transfer-code
python -m pytest browser_preflight.py -q
```

Run this transfer's tests in Terminal B. Stop this Terminal A server before the next workspace; state from the demonstration is not transferred.

To resume either transfer, repeat its host-side entry command without `abb prepare`. Do not overwrite an existing workspace.

Start with the correct application and case.json. Transfer input includes the valid endpoint 65535. The facilitator then restarts with `--variant bad-boundary`, using the same clean-state procedure. The agent must detect the requirement violation from test evidence.

Transfer general runners/helpers, not a finished demo test file. Keep the variant label undisclosed until evaluation when feasible. The supplied source includes the variants for reproducibility; this is a classroom exercise, not an adversarially isolated benchmark.

Acceptance: the improved suite passes correct behavior and detects the invalid rejection of a valid boundary for the relevant reason. It also meets the full required-case list on the preceding page. The facilitator reviews the actual assertions, not only pytest exit status.

Debrief: how did executable feedback expose a misleading green test result, and which parts of that evaluation still need human judgment?

8 Use local CI tools and separate grading

The current lightweight test workflow creates `.venv`, installs pinned Python dependencies and runs deterministic checks. See COPILOT_ACTIONS_GUIDE.md for the active workflow and real rehearsal instructions. Historical Actions records describe earlier runtime versions.

For a real rehearsal, provide the private `copilot_token` file through `ABB_SECRETS_DIR` or `.auth/secrets/`. The CLI imports it through stdin into a temporary profile. Run independent grading separately without provider credentials or access to secret files. A virtual environment isolates Python dependencies; OS accounts and filesystem permissions provide credential isolation.

Keep the 600-second stage timeout and engineering acceptance contracts. Rehearsal exit alone does not mean acceptance. Retain raw failures, assessments and FAIL/BLOCKED outcomes.

```
python -m training_runtime.ci --provider copilot -- python training_ci/run_lab.py fw1
python training_ci/grade_all.py fw1
```

8 Run and inspect the six-lab rehearsal

Workflow: `.github/workflows/validate-workflow-labs.yml`, named **Validate workflow training labs**. **Branch:** `training/workflow-labs`. It runs on changes to the training files on that branch. The workflow and materials are reviewable there; they have not been merged into main.

https://github.com/StefanTyllo/ABB-Training/tree/training/workflow-labs

The initial validation run is linked below. To repeat the same commit, open the run and choose **Re-run all jobs**. Also rerun the targeted follow-ups and package check linked in Section 10 to reproduce the reviewed result. A revised training commit triggers a new run. The workflow also defines workflow_dispatch; GitHub's manual dispatch UI requires the workflow to exist on the default branch, so do not depend on that UI before normal review and merge.

https://github.com/StefanTyllo/ABB-Training/actions/runs/38592278866

What each job does

1. Prepare three isolated workspaces: demonstration, fresh MD-only transfer, and fresh MD-plus-code transfer.
2. Invoke GitHub Copilot CLI in four ordered stages with focused prompts and a current-stage context file. The full original task card and local contracts remain unchanged; consult other card sections on demand. Each stage reads the previous saved artifacts: demonstrate/document, abstract, select deterministic work, implement verbose code. The task card supplies the scripted engineering rationale in CI; a human supplies it in class.
3. Preserve the MD-only checkpoint. Transfer only the selected workflow and, in code mode, reusable helpers. Do not transfer repaired parser sources, finished browser tests, device state, reports, or demo evidence.
4. Execute both fresh tasks with actual tools. Compare authoritative input hashes with the clean preparation snapshot.
5. Run independent Python/C/browser checks in a separate step without the Copilot token. These exit statuses determine assessment, not Copilot's completion message.

Independent checks: firmware grading verifies the original device baseline and one-change recovery, or rebuilds the parser for 50 stream cases. Hardware grading verifies authority, report arithmetic, calibration provenance and repeat-run stability. Browser grading uses real Chrome, exact persistence assertions, and relevant failures on deliberately faulty variants. All modes use the same task criteria; code mode additionally invokes the generated helpers.

Inspect and retain the evidence

Open each job's **Independently grade generated work** step and summary. Download its `lab-<id>-<attempt>` artifact before the 14-day retention expires. Inspect assessment.json, grader logs, stage prompts/transcripts, exact context snapshots/delivery hashes, execution/timeout metadata, generated workflows/code, and test evidence. A green agent step alone is insufficient. Browser startup failures and missing evidence are failures of readiness, not detected application defects.

Interpretation: this is a scripted Copilot rehearsal with separate stage invocations and file handoffs. It does not reproduce a human teaching conversation, validate every participant account, establish classroom completion time, or prove MD-plus-code is statistically superior. Use the facilitator rubric and learner pilot for those remaining training judgments.

9 Assess the learning outcome

The facilitator grades each lab using the same rubric. The engineering result and the training result are separate: correctly reporting a blocked source or a failed voltage limit can earn full training credit.

Criterion	0 points	1 point	2 points
Demonstration evidence	No actual command record	Partial record or unexplained decisions	Commands, observations, decisions, and why are traceable
Reusable workflow	Incident copied verbatim	Some parameters; missing branch or input	Defined inputs, branches, completion, and no demo-only constants
Deterministic boundary	Everything delegated to model	Some useful extraction; judgment blurred	Repeatable operations in code; judgment explicitly retained
Code quality	Opaque or unverified	Readable but incomplete failures/checks	Descriptive, explanatory code with valid and failure checks
Feedback and transfer	Unsupported success claim	Partial transfer evidence	Correct fresh-case result with actionable feedback

Training pass: at least 8/10, no zero in any row, and every mandatory task-specific acceptance condition satisfied. An edited authoritative source, fabricated observation, or copied demo solution instead of workflow transfer prevents a pass regardless of score. For test-track labs, a missing real-browser run also prevents a pass. This is the workshop's proposed rubric, not an external certification.

Feedback procedure: identify the failed criterion, point to the exact missing or incorrect evidence, request one focused revision, and rerun the relevant check. Do not repeatedly rerun unrelated checks after the issue has been resolved.

Submission record

- Participant/team, lab ID, agent/model and CLI version.
- Paths to the two workflow checkpoints, extraction map, automation, and evidence.
- Demonstration outcome and actual verification commands.
- Transfer outcome: accepted, unresolved, or blocked, with the reason.
- Rubric scores, mandatory criteria, required revision, and reassessment result.

Final participant explanation: choose one operation and show its progression from demonstrated commands, to Markdown instruction, to deterministic code, to a real failure message and recovery. Explain one decision that appropriately remained outside deterministic code.

Next use at work: select one recurring team task; run the same four stages with its actual requirements and authorized tools. Replace the synthetic adapter deliberately and validate it before relying on captured device or document data.

10 Facilitator rehearsal and release record

Before teaching, complete a full rehearsal on the participant environment. Run the same PDF extraction path, commands, account login, browser, and compiler used by learners. Record versions and outcomes. A checklist without the underlying run evidence is incomplete.

Starter checks, from the training root Host terminal — facilitator package only:

```
python -m pytest tests/runtime -q
```

The facilitator's independent checks and assessment sources are supplied separately from the learner materials.

These verify supplied fixture behavior. They do not replace learner code review or the live browser/agent checks. The hardware README documents its report checker; firmware smoke also grades a submitted parser executable.

Release check	Required result
Instruction walkthrough	A reviewer follows setup and a lab without inventing a path, command, input, or success condition
Fresh preparation	All six demo and transfer cases prepare without modifying clean originals
Known failures	Seeded parser/data/app defects exist and are detected by appropriate independent checks
GitHub Copilot CLI	Actual smoke test passes with the intended account and recorded version/model
Browser	Preflight and correct-app tests pass; strengthened tests detect both app defects for the right reason
Pedagogy	Demonstration precedes extraction; abstraction precedes reusable automation
Pilot	One representative learner completes each team's first lab and understands the handoff to its second
PDF	All pages readable; commands copy correctly; attachment extracts and matches its recorded digest

Historical verification before Docker. All twelve clean lab/case preparations, firmware reference checks, hardware fixture checks and HTTP variants passed. Every PDF page was visually inspected and the embedded ZIP was extracted and exercised. Authenticated Copilot Actions rehearsals, real Chrome checks, and targeted refinements are recorded on the following page and in the attachment's VERIFICATION.md. Codex account access and a representative learner pilot were not performed by CI.

Release decision to complete locally: READY / BLOCKED. Record date, environment, reviewer, evidence location, unresolved issue, and owner. Mark READY only after every applicable row passes. This keeps “training accepted” tied to observed readiness rather than the appearance of the document.

Known boundaries: the learner archive and PDF attachment omit assessment sources; the facilitator edition contains them, and the workshop remains a classroom assessment rather than an adversarial isolation benchmark; the fixtures are single-user and sequential; source-format extraction, physical hardware, and live Azure DevOps are later extensions. Do not infer coverage of those integrations from these exercises.

10 Observed CI results and refinements

Historical host-based environment (not Docker evidence): GitHub-hosted Ubuntu; Node.js 22; Python 3.12.15; Copilot CLI 1.0.97; repository secret COPILOT. The CLI default model was used; its exact model identity was not pinned or captured. These are functional rehearsal results, not a controlled model-performance study.

Initial six-lab run: commit `e36498d5e48667077444be81c6061a6dacd11bd6`. Four stage invocations and two fresh transfer invocations ran per lab. Independent graders executed separately without the Copilot token.

https://github.com/StefanTyllo/ABB-Training/actions/runs/38592278866

Lab	Demonstration	MD-only transfer	Code-assisted transfer
FW1	Pass	Pass	Pass after targeted refinement
FW2	Pass	Pass	Pass
HW1	Pass	Accepted blocked	Accepted blocked
HW2	Pass	Accepted design fail	Accepted design fail
T1	Pass	Pass	Pass; reporting refinement below
T2	Pass	Pass	Pass after timeout correction

Edition 2 prompt update. The focused context/runner changes in this edition have token-free regression and material checks. The authenticated runs summarized here used the earlier prompts; they are historical evidence, not measured outcomes for the new delivery. Run a new authenticated Copilot rehearsal before claiming new agent acceptance.

How to read this table. “Accepted blocked” means missing source authority was correctly detected; “accepted design fail” means the unsafe design was accurately reported. Neither means the underlying engineering item passed. Code-assisted outcomes include the explicitly identified follow-ups on the next page.

Evidence location. On the training branch, open validation_records/README.md for archive-to-run and commit mappings. archive_manifest.json contains byte counts and SHA-256 hashes. The exact original Actions archives are retained with the corrections; the initial six-job run remains failed because of the T2 timeout.

https://github.com/StefanTyllo/ABB-Training/tree/training/workflow-labs/validation_records

Use in training. Copy the student starters from this PDF. Completed CI-generated code is review evidence; giving it to learners before their demonstration would bypass the workflow-extraction exercise.

Review findings and corrections

FW1: the initial evaluator accepted the final state, but the transcript showed a denied compound shell command and manual fallback. A fresh follow-up corrected default raw-capture retention, used atomic helper invocations, made the one change through code, and independently verified before/after captures, mutation evidence and byte-preserved reruns. Its transcript records 18 passing helper tests.

https://github.com/StefanTyllo/ABB-Training/actions/runs/38594335199

T1: initial real-Chrome checks verified visibility and exact persistence, and the code transfer actually invoked unchanged helpers. Review found a null browser field and temporary-only screenshots. A targeted follow-up adds actual browser capabilities, retained missing-element evidence, empty-password rejection and invalid-save preservation. The follow-up passed. A complete-package check restored four original XML fixtures omitted from the refinement seed, then passed all 13 helper self-tests and the eight browser tests.

https://github.com/StefanTyllo/ABB-Training/actions/runs/38595251454

Complete T1 package verification: https://github.com/StefanTyllo/ABB-Training/actions/runs/38596385772

T2: the initial demo and MD-only transfer passed. Code transfer hit the outer 300-second grader limit after its direct browser checks passed. The unchanged generated package was regraded with a 900-second limit; assertions and protected-input checks are unchanged. The full package passed: 16 helper tests, 12 correct-app browser cases, relevant failures for both mutants and protected-input hashes.

https://github.com/StefanTyllo/ABB-Training/actions/runs/38597697875

Hardware: both transferred calculation/checker implementations worked. Their test suites initially assumed demo inputs and were repaired into independent fixtures during transfer. The handbook now explicitly requires fixtures independent of the active case. Do not describe those transfers as zero-repair portability.

Interpretation: T2 reports mark inferred failure phases for review. The valid 65535 rejection occurs at Save, before reload; the demo heuristic labels it after reload. Use the assertion evidence to determine the phase.

Evidence and limits: original run archives and review notes are retained under validation_records/ on the training branch, with hashes. The six synthetic tasks exercise the stated paths; they do not establish production-device readiness, comprehensive adversarial resistance or improved learning outcomes. Use the local account/browser gates and representative learner pilot before classroom acceptance.

11 Official setup references

Verified against official documentation on 14 October 2026. Commands and policies can change; rerun the live readiness checks before delivery. The synthetic lab requirements are supplied with the attachment and are the authority for exercise behavior.

G1 GitHub Copilot CLI installation https://docs.github.com/en/copilot/how-tos/copilot-cli/set-up-copilot-cli/install-copilot-cli

G2 GitHub Copilot CLI authentication https://docs.github.com/en/copilot/how-tos/copilot-cli/set-up-copilot-cli/authenticate-copilot-cli

G3 Copilot CLI programmatic reference https://docs.github.com/en/copilot/reference/copilot-cli-reference/cli-programmatic-reference

G4 Authentication troubleshooting https://docs.github.com/en/copilot/how-tos/copilot-cli/set-up-copilot-cli/troubleshoot-copilot-cli-auth

G5 Copilot CLI and GitHub Actions with personal-token setup https://docs.github.com/en/copilot/how-tos/copilot-cli/automate-copilot-cli/automate-with-actions

G6 Copilot CLI in Actions with GITHUB_TOKEN https://docs.github.com/en/copilot/how-tos/copilot-cli/use-copilot-cli-in-actions

G7 Official setup-node action and workflow examples https://github.com/actions/setup-node

S1 Selenium Page Object guidance
https://www.selenium.dev/documentation/test_practices/encouraged/page_object_models/

S2 Selenium wait guidance https://www.selenium.dev/documentation/webdriver/waits/

Software download entry points for the facilitator https://nodejs.org/en/download
https://www.python.org/downloads/ https://git-scm.com/downloads https://www.google.com/chrome/

The four-stage training method comes from the user's demonstrated-workflow approach. The rubric, fixtures, prompts, and replay procedure are authored training designs. No claim of published validation of these six specific labs is made.

12 Research basis and limits of the evidence

The shared hypothesis is that engineers can express a repeatable process as a workflow, then replace its fully specified operations with code that gives the agent observable feedback. The papers below support parts of this design. They do not validate these ABB exercises or establish that code extraction always outperforms a well-written Markdown workflow.

Publication evidence	Training design implication
Yang et al., SWE-agent, NeurIPS 2024: the interface through which an agent edits files and runs programs affects repository-task performance. [R1]	Give repeated operations explicit commands, bounded inputs and interpretable results. This is the rationale for capture/build/check entry points.
Shinn et al., Reflexion, 2023: agents can use task feedback and retained textual reflection to improve later attempts in studied environments. [R2]	Preserve observations and explanations before abstraction. Use actual execution feedback in the next decision; do not merely ask for another confident answer.
Olausson et al., Is Self-Repair a Silver Bullet for Code Generation?, ICLR 2024: repair gains can be modest, variable or absent when cost is considered; feedback quality matters. [R3]	Keep independent checks, compare fresh cases, record effort and failure reasons, and retain human engineering review. More repair iterations are not automatically better.

Our inference: explicit deterministic collectors and evaluators can reduce the amount of routine reconstruction requested from the model, while making some process deviations detectable. This is a design argument to test, not a published causal result for the four-stage training method.

What to measure in these labs: the same task criteria for both replay conditions; source preservation; correct handling of missing authority and failed limits; use of actual tool output; assertion-based detection of seeded defects; runtime and interventions. A single pair can expose a defect in the workflow, but cannot estimate a reliable success-rate difference. CI stage invocations also differ from the classroom's continuous demonstration conversation.

R1 John Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024. Official proceedings: https://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html

R2 Noah Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. Author manuscript, version 4, 10 October 2023: https://arxiv.org/abs/2303.11366

R3 Theo X. Olausson et al. Is Self-Repair a Silver Bullet for Code Generation? ICLR 2024. Official proceedings: https://proceedings.iclr.cc/paper_files/paper/2024/hash/9ddc141bdbf9d1db510cefff56c586ad-Abstract-Conference.html

13 Stochasticity: one prompt, several valid answers

Use before the four-stage method. This supplement explains why we extract repeatable operations into code. Read it before Section 5 and revisit it when comparing fresh-case replays in Section 7.

What stochasticity means. Language-model generation is probabilistic. For many prompts there is no single uniquely correct next token, so separate runs can choose different likely continuations and still satisfy the request. Wording, ordering, examples, level of detail, and sometimes conclusions can vary.

Same request, different useful answers

> Plan for me a trip to Thailand for a 2 week vacation, what do you recommend for sightseeing?

Run	Possible opening or structure
1	Bangkok first: Grand Palace and Wat Phra Kaew, then Wat Arun, northern cultural sites, and beaches.
2	Starts by asking about interests, then groups recommendations by culture, temples, beaches, and islands.
3	Bangkok, Chiang Mai / northern Thailand, then southern islands, presented as a day-by-day itinerary.

These are illustrative alternatives from the supplied training file, not measured runs or travel recommendations. Different wording or structure alone does not establish that an answer is wrong; check it against the request and evidence.

What this means for the ABB labs

The following engineering examples adapt the source's principles to this training. Two runs may choose different troubleshooting hypotheses, test cases, or report structures. Judge each against the same requirements and observed results. An unsupported conclusion is still an error, even if the text sounds confident.

Demonstration. Repeat the travel prompt in three fresh conversations using the same model and settings. Compare structure, assumptions, and coverage. Identify which differences are harmless and which would require clarification or verification. Treat the examples above as possibilities, not expected outputs.

Bridge to workflow extraction. A Markdown workflow gives the model guidance, but leaves it to interpret and execute steps again. Move fully specified parsing, calculations, comparisons, and checks into reviewed code; keep uncertain diagnosis and engineering decisions with the model or engineer.

Source: supplied `chatgpt-astra-training.html`, module "Stochasticity: One Prompt, Several Valid Answers". The three-run demonstration and ABB applications are adaptations for this handbook.

13 Reduce unwanted variability

Aim. Make required operations and acceptance checks repeatable while retaining useful model judgment. Clearer prompts reduce ambiguity; they do not guarantee identical answers or correct conclusions.

Technique from the source	How to use it in these labs
Atomic prompts	Ask for one bounded task at a time. First capture state; then compare it with the identity-matched reference; only then propose a change.
Constrain the output	Specify inputs, authoritative sources, required fields, ordering, exclusions, and acceptance criteria. Example: report expected and observed values, evidence paths, and pass / fail / blocked.
Separate deterministic and generative steps	Let the model propose a hypothesis. Use reviewed code for parsing, dates, totals, unit conversions, limit checks, and rule compliance when inputs and rules are fixed.
Repeat and compare when stakes justify it	Run an analysis independently again, compare material differences, and verify them against authoritative evidence. Repeated agreement alone is not verification.

Connect the techniques to the four stages

1. Demonstrate and document. Use one bounded operation per prompt. Record the actual command, observation, reasoning, and recovery so a later answer cannot silently rewrite the history.

2. Abstract the workflow. Define inputs, branches, source authority, missing-input behavior, and completion criteria. Replace case-specific values with explicit parameters.

3. Choose the deterministic boundary. Separate operations with fully specified rules from diagnosis, ambiguous requirements, and decisions requiring human authority.

4. Implement and verify. Turn those specified operations into readable helpers with explicit inputs, outputs, failures, and independent checks. The same code and fixed inputs should give repeatable results for these operations; devices, clocks, files, and other changing state still need to be controlled or recorded.

Example: HW2 calibration. The model helps interpret the requirement and explain a failed limit. Reviewed code calculates the values, checks units and limits, and detects stale calibration from the actual source inputs. A fluent explanation cannot replace those checks.

Replay debrief. Use the same model, task, and acceptance criteria for MD-only and code-assisted runs. Record actual helper use, changing inputs, repairs, and interventions. Compare evidence rather than prose similarity. A single successful pair does not establish a reliable success-rate difference.

Takeaway. Use the model for interpretation and judgment; use verified deterministic operations wherever the rules and inputs are fully specified.

Source: supplied `chatgpt-astra-training.html`, slide "Reduce Unwanted Variability". Lab examples and the four-stage mapping are handbook adaptations.